

社交新零售多模式电商系统 · 技术文档

版本 v1.0 · 最后更新 2026-09

本文档供团队内部与新入职工程师阅读，覆盖系统架构、核心模块设计、数据库结构与关键技术决策。

1. 项目背景

本项目是一套面向多客户定制开发的社交新零售电商系统，以微信小程序为前端载体，配套Web端运营后台。系统支持多种交易场景，包括：

- 多人拼团
- 商户独立小店（多商户入驻）
- 用户排位抢购
- 寄售排队转让

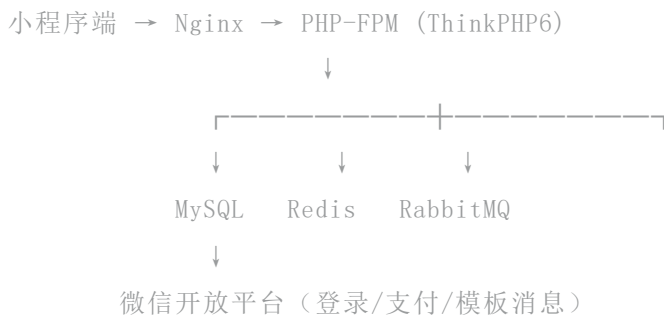
服务对象为快消、美妆、农产品等行业的中小品牌方，按客户需求做二次定制。

2. 技术栈

层	选型
后端框架	ThinkPHP 6 (PHP 8.1)
前端后台	Vue 2 + Element UI
小程序端	微信原生小程序（原生框架，非Taro/uni-app）
数据库	MySQL 8.0（主从，一主一从）
缓存	Redis 6.0（String/Hash/ZSet/分布式锁）
消息队列	RabbitMQ（异步下单、退款通知、微信支付回调补偿）
服务器	阿里云ECS 4核8G × 2台，Nginx负载均衡
部署	宝塔面板手动部署，未容器化

3. 系统架构

系统采用经典单体分层结构：



后台管理系统 (Vue + Element) 通过同一套PHP接口服务，按RBAC权限区分运营、商户、超级管理员三类角色。

4. 核心业务模块

4.1 多人拼团

业务规则：

- 拼团人数分 2人团、3人团、5人团三档，由运营在后台配置
- 成团时限为 24 小时，超时未成团系统自动关闭订单并原路退款
- 拼团价格固定，不支持优惠券叠加
- 团长免单活动单独配置，与普通拼团互斥

技术实现：

- 拼团活动表 `group_activity`，开团订单表 `group_order`
- 用 Redis ZSet 存储待成团订单，score为成团截止时间戳
- 每天凌晨由定时任务扫描超时团，调用微信支付退款接口
- 防止超卖：下单时先 Redis DECR 库存，扣减成功才创建订单；失败则回滚库存

4.2 商户独立小店

业务规则：

- 商户入驻需提交营业执照、法人身份证、对公账户
- 审核通过后开通独立小店，可上架自己的商品
- 订单货款 T+7 结算，扣除平台佣金（默认5%）后打款到商户对公账户
- 商户只能看到自己店铺的订单和客户

技术实现：

- 商户表 `merchant` , 商品表 `goods` 带 `merchant_id` 外键
- 所有查询强制加 `merchant_id` 条件, 在BaseModel里做全局作用域
- 结算走微信支付"商家转账到零钱"API, T+7由定时任务批量执行

4.3 用户排位抢购

业务规则:

- 限量商品按排位顺序购买, 先排队先得
- 每位用户同一活动只能占一个排位
- 排位有效期 15 分钟, 未支付自动释放

技术实现:

- Redis List 模拟队列, `LPUSH` 入队、`RPOP` 出队
- 用 Hash 记录用户占位: `rank:{activity_id}:{user_id} → expire_at`
- 过期监听: Redis Keyspace Notifications 监听过期事件, 自动释放排位

4.4 寄售排队

业务规则:

- 用户买入商品后可选择"寄售"挂单转让给下一位用户
- 寄售价格由系统按规则自动上浮 5%, 不允许用户自定价
- 成交后系统扣除 3% 手续费, 剩余货款打回原用户钱包

技术实现:

- 寄售单表 `consign_order` , 状态机: 挂单中 → 已匹配 → 已成交 → 已退款
- 撮合逻辑: 新买单进来时, 按挂单时间最早优先匹配
- 钱包流水表 `wallet_log` , 所有余额变动必须留痕, 禁止直接改余额字段

5. 数据库设计要点

5.1 订单表 `order`

字段	类型	说明
<code>id</code>	<code>bigint unsigned</code>	主键
<code>order_no</code>	<code>varchar(32)</code>	订单号, 年月日+随机串, 唯一索引

user_id	bigint	下单用户
merchant_id	bigint	所属商户
total_amount	decimal(10,2)	订单总额
status	tinyint	0待付款 1已付款 2已发货 3已完成 4已退款
pay_time	datetime	支付时间
created_at	datetime	

索引： (user_id, status) 、 (merchant_id, status) 、 order_no 唯一索引。

5.2 关键设计原则

- 所有金额字段用 decimal(10,2) ， 禁止 float
- 订单号在应用层生成，不依赖自增ID
- 软删除用 deleted_at 字段，不物理删除
- 大表（订单、钱包流水）按月分表，目前数据量在百万级，暂未分库

6. 缓存与队列使用场景

6.1 Redis

场景	数据结构	Key示例
商品详情缓存	String	goods:detail:{goods_id} ， TTL 1小时
购物车	Hash	cart:{user_id}
拼团待成团列表	ZSet	group:waiting:{activity_id}
排队队列	List	rank:queue:{activity_id}
接口限流	String + 过期	rate:limit:{user_id}:{api}
分布式锁	String	lock:order:{order_no} ， TTL 10秒

6.2 RabbitMQ

队列	用途
----	----

order_created	下单成功后异步发短信/模板消息
order_paid	支付成功后扣库存、加积分、通知商户
refund_notify	退款结果异步通知用户
stat_daily	每日凌晨统计订单数据，避免实时算聚合

7. 微信生态对接

7.1 微信登录

- 小程序端 `wx.login()` 拿 `code` → 后端调 `auth.code2Session` 换 `openid`
- 后端签发自定义JWT，7天有效期，存在小程序Storage

7.2 微信支付

- 统一下单用 V3 接口
- 支付结果通过回调URL异步通知，**同时**提供主动查询接口做兜底
- 回调接口做了幂等处理：用 `out_trade_no` 加分布式锁，重复回调直接返回成功
- 退款走退款API，结果同样异步通知

7.3 微信模板消息

- 下单成功、发货、退款结果三类通知
- 模板ID在后台配置，支持按商户切换

8. 遇到的问题与优化

8.1 问题：拼团接口偶发超卖

现象：秒杀类拼团活动，QPS 200 时偶尔出现超卖。

原因：MySQL 行锁在高并发下排队太长，部分请求超时后重试。

解决：

1. 库存扣前移到 Redis：下单前先 `DECR stock:{goods_id}`
2. Redis 扣减成功才写订单
3. 异步通过 RabbitMQ 同步数据库库存
4. 失败回滚 Redis 库存

优化后超卖问题消失，接口 P99 从 800ms 降到 200ms。

8.2 问题：商户结算批量超时

现象：T+7 批量结算，商户数多时脚本跑超过 30 分钟。

解决：

1. 结算单分片，每 100 个商户一个任务
2. 通过 RabbitMQ 分发，多消费者并发处理
3. 失败重试 3 次，仍失败进死信队列人工介入

8.3 问题：小程序包体积超限

现象：主包超过 2MB 限制。

解决：

1. 把拼团、寄售等不常用页面拆成分包
2. 图片走CDN，不打包进小程序
3. 最终主包压到 1.4MB

9. 部署与运维

- 线上两台ECS，Nginx做负载，会话存Redis不依赖单机
- 数据库每日凌晨全量备份到OSS，保留30天
- 日志按月切割，用ThinkPHP内置文件日志
- 暂无监控告警，依赖用户反馈发现问题（待改进项）

10. 待改进项

1. **容器化：**目前手动部署，计划迁移到Docker Compose
2. **监控：**接入Prometheus + Grafana，监控接口耗时和错误率
3. **测试：**核心下单流程缺自动化测试，目前靠回归
4. **ES搜索：**商品搜索目前用MySQL LIKE，数据量大后需要迁到Elasticsearch
5. **CI/CD：**目前手动git pull + 重启PHP-FPM，需自动化

常见问题（FAQ）

Q: 拼团多久未成团自动退款?

A: 24小时。

Q: 商户结算周期是多久?

A: T+7, 订单完成后第7天打款到商户对公账户。

Q: 平台佣金默认比例?

A: 5%, 可在后台按商户单独配置。

Q: 排位有效期多久?

A: 15分钟, 未支付自动释放。

Q: 寄售价格谁定?

A: 系统按规则自动上浮5%, 用户不能自定义。

Q: 寄售手续费多少?

A: 成交金额的3%。

Q: 用了什么消息队列?

A: RabbitMQ, 主要用于下单通知、支付回调、退款通知、每日统计。

Q: 怎么防止超卖?

A: Redis预扣库存 + RabbitMQ异步落库, 数据库层做兜底乐观锁。

Q: 数据库有没有分库分表?

A: 暂未分库, 订单表按月分表。数据量百万级, 未来过千万再考虑分库。

Q: 部署方式?

A: 两台阿里云ECS + Nginx负载, 宝塔面板手动部署, 未容器化。